MPOO 2027-1 · GRUPO 6 · PRÁCTICA 8 · MIÉRCOLES 21 DE OCTUBRE

Herencia, segunda parte: la estación de clasificación

Una estación con una banda transportadora, tres sensores y tres actuadores. Primero la diseñas en un **diagrama de clases**; luego la programas en cuatro paquetes con lo del lunes: `abstract`, `this(...)`, `super(...)`, sobrecarga, sobrescritura, modificadores, composición y agregación.

Descargar los archivos (.zip)

En una línea

Dibujas el diagrama, escribes once clases y terminas cuando `Main` imprime **exactamente** lo que dice esta guía.

QUÉ	QUIÉN	DÓNDE VA
Diagrama de clases (parte 1)	Tú, en pareja, antes de programar	Papel o draw.io; va en la bitácora
`Luz.java`, `PruebaTemperatura.java`, `Main.java`	Vienen en el .zip. **No se tocan**	`dispositivos` y `estacion`
`Bateria`, `Dispositivo`, `Sensor`, `SensorTemperatura` (parte 2)	Tú, en pareja	`dispositivos` y `sensores`
`SensorPresencia`, `SensorPeso`, `Actuador`, `BandaTransportadora`, `Semaforo`, `Alarma`, `Estacion` (parte 3)	Tú, en pareja	`sensores`, `dispositivos`, `actuadores` y `estacion`

`Main.java` se agrega **hasta el final de la parte 3**: IntelliJ compila todo lo que hay en `src` y, si lo pones antes, nada corre porque usa clases que todavía no existen.

Parte 0 · Qué se entrega y cómo se califica

QUÉ	DETALLE
Cuándo	**Viernes 23 de octubre, 23:59.** Se empieza en el laboratorio y ahí mismo se resuelven las dudas. Sin IA, como siempre.
Con quién	En parejas, o de tres si alguien se queda sin pareja. **Cada integrante sube la entrega completa en su propia carpeta**.
Dónde	Tu rama `entregas_apellido_nombre`, carpeta `entregas/apellido_nombre/p08/`. Solo push, sin pull request.
Qué archivos	La carpeta `src` completa con sus **cuatro paquetes** (los catorce `.java`), **todos** los `.class` y una **bitácora en PDF** con: los integrantes (nombre y número de cuenta), **tu diagrama de clases** (foto o captura), la **captura de la salida** de `Main`, la **parte 4** resuelta y qué se les complicó.

Se califica leyendo tu código y tu diagrama, sobre 10:

QUÉ REVISO	PUNTOS
Diagrama de clases: las 12 cajas con `+ - # ~`, abstractas en cursiva, triángulos de herencia, ◆ composición, ◇ agregación y multiplicidad	2
Corre y la salida es la de la guía	2
Constructores: `super(...)` en la primera línea de cada hijo, y `this(...)` en los constructores sobrecargados de `Actuador` y `SensorPeso`	2
Sobrecarga y sobrescritura: `instalar` y `avanzar` sobrecargados; `@Override` en `medir`, `hayAlerta`, `reaccionar` y `toString`, con `super.toString()`	1.5
Modificadores y paquetes: atributos `private`, `protected` solo donde lo pide la guía, cada clase en su paquete con su `package` e `import`	1.5

Bitácora: la parte 4 explicada con tus palabras y con tu código	1

El código tiene que ser suyo. Si algo no les sale, entréguenlo igual y cuenten en la bitácora dónde se atoraron.

El enunciado

En el **Andén 3** de una paquetería hay una **estación de clasificación**. La estación tiene un nombre y tiene instalados **uno o más sensores** y **uno o más actuadores** (le caben hasta cinco de cada uno).

Todo **dispositivo** tiene un identificador que nunca cambia (`S1`, `A2`…) y **su propia batería**, que se crea con él y se gasta con el uso. No existe un "dispositivo" a secas: es un **sensor** o un **actuador**.

Un sensor recibe una señal, la **mide** en su unidad y dice si hay **alerta**. Hay tres: el de **presencia** (la distancia al paquete, en cm; alerta si está atorado a menos de 2 cm), el de **peso** (en kg; alerta si pasa su límite) y el de **temperatura** del motor (en grados, con una calibración; alerta si pasa su máxima).

Un actuador se enciende y se apaga, y **reacciona** a lo que dicen los sensores. Hay tres: la **banda transportadora** (avanza o se detiene), el **semáforo** (tiene tres **luces**, roja, amarilla y verde, que se fabrican con él) y la **alarma** (suena a un volumen de 0 a 10).

En cada ciclo, la estación lee todos sus sensores; si **alguno** da alerta, la banda se detiene, el semáforo se pone en rojo y suena la alarma. Si un sensor sale a mantenimiento, **sigue existiendo** y se puede volver a instalar.

El mapa: cómo debe quedar

Antes de dibujar o programar, ubícate. Así queda la estación cuando terminas.

La herencia y las relaciones

```
Dispositivo (abstract)        dispositivos  ◆ 1 Bateria
├── Sensor (abstract)         dispositivos
│   ├── SensorPresencia       sensores
│   ├── SensorPeso            sensores
│   └── SensorTemperatura     sensores
└── Actuador (abstract)       dispositivos
    ├── BandaTransportadora   actuadores
    ├── Semaforo              actuadores    ◆ 3 Luz
    └── Alarma                actuadores

Estacion   estacion      ◇ 1..5 Sensor y ◇ 1..5 Actuador
Bateria    dispositivos  ◆ la crea Dispositivo con new
Luz        dispositivos  ◆ las crea Semaforo con new
```

Cada `├──` es un **«es un»** (`extends`): un `SensorPeso` es un `Sensor`, que es un `Dispositivo`. Cada ◆ y ◇ es un **«tiene un»**, un atributo que es otro objeto. ◆ **composición**: el dueño hace el `new` y la parte muere con él. ◇ **agregación**: la parte llega de afuera (aquí, de `Main`) y sobrevive aunque la saquen.

Las carpetas y los archivos

```
src
├── dispositivos
│   ├── Bateria.java              parte 2
│   ├── Dispositivo.java          parte 2
│   ├── Sensor.java               parte 2
│   ├── Actuador.java             parte 3
│   └── Luz.java                  viene en el .zip
├── sensores
│   ├── SensorTemperatura.java    parte 2
│   ├── SensorPresencia.java      parte 3
│   └── SensorPeso.java           parte 3
├── actuadores
│   ├── BandaTransportadora.java  parte 3
│   ├── Semaforo.java             parte 3
│   └── Alarma.java               parte 3
└── estacion
    ├── Estacion.java             parte 3
```

```
    ├── PruebaTemperatura.java    viene en el .zip
    └── Main.java                 viene en el .zip, va al final
```

Cada archivo empieza con el `package` de su carpeta (por ejemplo `package sensores;`) y hace `import` de lo que usa de otro paquete. Si arrastras las clases en IntelliJ y aceptas el **Refactor**, él pone el `package` por ti.

El orden para construir

De arriba hacia abajo: nunca escribas una hija antes que su padre, ni la estación antes que lo que va a recibir.

1. **Parte 2:** `Bateria` → `Dispositivo` → `Sensor` → `SensorTemperatura`. Punto de control: corre `PruebaTemperatura`.
2. **Parte 3:** `SensorPresencia` y `SensorPeso` → `Actuador` → `Alarma`, `BandaTransportadora` y `Semaforo` → `Estacion` → agrega `Main` y córrelo.

Parte 1 · El diagrama de clases, antes de programar

En pareja, en papel o en draw.io. Tienen 15 minutos; luego se proyecta la solución y corrigen el suyo. **El diagrama corregido es su mapa para la parte 3.**

1. **Una caja por clase del mapa**: son 12 (sin `Main` ni `PruebaTemperatura`). Tres compartimentos: nombre, atributos y métodos. Las **abstractas** llevan el nombre en cursiva o `«abstract»`, y sus métodos sin cuerpo también en cursiva.
2. **La herencia**: una flecha con triángulo vacío de cada hija a su padre, siguiendo el árbol.
3. **La composición y la agregación**: ◆ lleno del lado del dueño en `Dispositivo` – `Bateria` y `Semaforo` – `Luz`; ◇ vacío del lado de `Estacion` hacia `Sensor` y hacia `Actuador`.
4. **La multiplicidad** en cada ◆ y ◇: `1`, `3`, `1..5`.
5. **Llena las cajas** con los atributos y métodos de las tablas de las partes 2 y 3, cada uno con su símbolo: `+` public, `–` private, `#` protected, `~` de paquete.

Parte 2 · Los cimientos

Primero los paquetes del mapa. En IntelliJ: clic derecho en `src` → **New → Package**, cuatro veces. Del .zip, copia `Luz.java` a `dispositivos` y `PruebaTemperatura.java` a `estacion`. `Main.java` espera hasta el final.

Las carpetas, como en el mapa.

Escribe estas cuatro clases, en este orden. Son la base de todo lo demás: cuando `PruebaTemperatura` imprima lo de abajo, pasas a la parte 3.

`Bateria`, en `dispositivos`: sin `public`, a propósito

QUÉ	CÓMO
Declaración	`class Bateria`, **sin modificador**: solo su paquete la ve. Sus métodos y su constructor, también sin modificador
Atributo	`int carga`, de 0 a 100
Constructor	`Bateria(int carga)`: menor que 0 se guarda 0; mayor que 100, 100
Métodos	`getCarga()`, y `descargar(int puntos)`, que resta sin bajar de 0 (pista: `Math.max`)

`Dispositivo`, en `dispositivos`: abstracta, con composición y `this(...)`

QUÉ	CÓMO
Declaración	`public abstract class Dispositivo`
Atributos	`final String id` y `final Bateria bateria`
Constructores	**Dos.** `Dispositivo(String id, int carga)` guarda el id y hace el `new Bateria(carga)` **ahí adentro**: eso es composición. `Dispositivo(String id)` no repite nada: llama al otro con `this(id, 100)` en su primera línea
Métodos	`getId()`; `getCarga()`, que le pregunta a su batería; y `gastar(int puntos)`, `protected`: solo las hijas gastan batería

`toString()`	`"S3 (100%)"`: el id y la carga

`Sensor`, en `dispositivos`: abstracta, hija de otra abstracta

QUÉ	CÓMO
Declaración	`public abstract class Sensor extends Dispositivo`
Atributos	`final String unidad` y `double senal`
Constructores	**Dos.** `Sensor(String id, String unidad, int carga)` con `super(id, carga)`, y `Sensor(String id, String unidad)` con `this(...)` y la batería en 100
Métodos	`getUnidad()`; `recibir(double senal)`, con el que `Main` simula la señal del mundo real; y `getSenal()`, **`protected`**, porque solo las hijas leen la señal cruda, y viven en otro paquete
Abstractos	`double medir()` y `boolean hayAlerta()`, sin cuerpo: cada sensor pone el suyo
`leer()`	**`public final`**: ninguna hija lo cambia. Gasta 1 de batería y devuelve `"S3 mide 40.0 C"` (id, `" mide "`, `medir()`, un espacio y la unidad), con `" ALERTA"` al final si `hayAlerta()`
`toString()`	`"Sensor "` más lo del padre

`SensorTemperatura`, en `sensores`: la primera hoja

QUÉ	CÓMO
Declaración	`public class SensorTemperatura extends Sensor`. Vive en otro paquete: necesita `import dispositivos.Sensor;`
Atributos	`double calibracion` y `double maxima`
Constructor	`SensorTemperatura(String id, double calibracion, double maxima)`, con unidad `"C"` y la batería en 100

`medir()`	La señal más la calibración
`hayAlerta()`	Si mide más que su máxima
`toString()`	Lo del padre más `" temperatura, max 45.0"`

Con las cuatro listas, corre `PruebaTemperatura`:

```
package estacion;

import sensores.SensorTemperatura;

public class PruebaTemperatura {
    public static void main(String[] args) {
        SensorTemperatura termo = new SensorTemperatura("S3", -1.5,
45);
        System.out.println(termo);
        termo.recibir(41.5);
        System.out.println(termo.leer());
        termo.recibir(50);
        System.out.println(termo.leer());
        System.out.println(termo);
    }
}
```

> **DEBE SALIR**
>
> ```
> Sensor S3 (100%) temperatura, max 45.0
> S3 mide 40.0 C
> S3 mide 48.5 C ALERTA
> Sensor S3 (98%) temperatura, max 45.0
> ```

La última línea dice `98%`: cada `leer()` gastó un punto de la batería que el sensor trae adentro. Y `getSenal()` funcionó desde `sensores`, otro paquete, porque `SensorTemperatura` es hija.

Parte 3 · La estación completa

Mismo patrón que la parte 2. Todos los atributos son `private`. Las clases y lo que se usa desde otro paquete, `public`.

Los otros dos sensores, en `sensores`

CLASE	CONSTRUCTOR	`MEDIR()`	`HAYALERTA()`	`TOSTRING()`
`SensorPresencia`	`SensorPresencia(String id)`. Unidad `"cm"`, batería al 100	La señal llega en milímetros: la divide entre 10	Si mide menos de 2	Lo del padr más `" presencia'`
`SensorPeso`	**Dos**: `SensorPeso(String id, int carga, double limite)`, unidad `"kg"`, y si el límite es 0 o menos se guarda 1. Y `SensorPeso(String id, double limite)`, que con `this(...)` deja la batería en 100	La señal llega en gramos: la divide entre 1000	Si mide más que su límite	Lo del padr más `" peso, limite 25.0"`

`Actuador`, en `dispositivos`

QUÉ	CÓMO
Declaración	`public abstract class Actuador extends Dispositivo`
Atributo	`boolean encendido`, nace apagado
Constructores	**Dos**, como `Dispositivo`: `Actuador(String id, int carga)` y `Actuador(String id)`, que usa `this(...)` para dejar la batería en 100
Métodos	`isEncendido()`, `encender()`, `apagar()` y el abstracto `String reaccionar(boolean alerta)`

`toString()`	`"Actuador "` más lo del padre

Los tres actuadores, en `actuadores`

CLASE	CONSTRUCTOR	LO SUYO	`REACCIONAR(ALERTA)`	TOS
`BandaTransportadora`	`(String id, int velocidad)`; velocidad menor que 1 se guarda 1	`int recorrido` con su getter. **Sobrecarga**: `avanzar(int metros)` suma los metros, gasta esa batería y devuelve `"A1 avanza 2 m, lleva 2 m"`, o `"A1 apagada, no avanza"` si está apagada. `avanzar()` avanza su velocidad llamando a la otra	Con alerta: se apaga y devuelve `"A1 se detiene"`. Sin alerta: se enciende y devuelve lo de `avanzar()`	Lo más ban por
`Semaforo`	`(String id)`. Crea sus tres `Luz` en un arreglo: rojo, amarillo y verde (**composición**)	`ponerEn(String color)` enciende esa luz y apaga las otras. `getColor()` devuelve el color de la encendida, o `"apagado"`	Se enciende; rojo con alerta, verde sin ella. Devuelve `"A2 en verde"`	Lo más sem en

`Alarma`	`(String id, int volumen)`; el volumen se queda entre 0 y 10	`getVolumen()`	Con alerta: se enciende y devuelve `"A3 suena a volumen 10"`. Sin alerta: se apaga y devuelve `"A3 en silencio"`	Lo más `ala vol 10"`

`Estacion`, en `estacion`

QUÉ	CÓMO
Atributos	`final String nombre`; `Sensor[] sensores` y `Actuador[] actuadores` de tamaño 5, con sus contadores `numSensores` y `numActuadores`; y `int ciclos`
Constructor	`Estacion(String nombre)`: guarda el nombre y crea los dos arreglos **vacíos**. Los sensores no se crean aquí: llegan de afuera (**agregación**)
`instalar(Sensor s)` y `instalar(Actuador a)`	**Sobrecarga.** Lo guarda en el siguiente lugar libre e imprime `"Instalado: "` más el dispositivo. Si ya no cabe, imprime `"Sin lugar para S1"`
`Sensor retirar(String id)`	Lo busca por id, recorre los que siguen un lugar a la izquierda, imprime `"Retirado: S2"` y **devuelve el sensor**. Si no existe, imprime `"No existe el sensor S9"` y devuelve `null`
`ciclo()`	Suma un ciclo e imprime `"== Ciclo 1 en Anden 3"`. Luego imprime `" "` más el `leer()` de cada sensor; si alguno `hayAlerta()`, hay alerta. Al final imprime `" "` más el `reaccionar(alerta)` de cada actuador
`toString()`	`"Anden 3: 3 sensores, 3 actuadores, 3 ciclos"`

Con todo listo, agrega `Main.java` a `estacion` y córrelo:

```
package estacion;

import actuadores.Alarma;
import actuadores.BandaTransportadora;
import actuadores.Semaforo;
import dispositivos.Sensor;
import sensores.SensorPeso;
import sensores.SensorPresencia;
import sensores.SensorTemperatura;

public class Main {
    public static void main(String[] args) {
        // los dispositivos nacen aquí, fuera de la estación
        SensorPresencia ojo = new SensorPresencia("S1");
        SensorPeso bascula = new SensorPeso("S2", 80, 25);
        SensorTemperatura termo = new SensorTemperatura("S3", -1.5,
45);
        BandaTransportadora banda = new BandaTransportadora("A1", 2);
        Semaforo semaforo = new Semaforo("A2");
        Alarma alarma = new Alarma("A3", 15);

        Estacion anden = new Estacion("Anden 3");
        anden.instalar(ojo);
        anden.instalar(bascula);
        anden.instalar(termo);
        anden.instalar(banda);
        anden.instalar(semaforo);
        anden.instalar(alarma);

        // un paquete normal
        ojo.recibir(120);
        bascula.recibir(18500);
        termo.recibir(41.5);
        anden.ciclo();

        // un paquete con sobrepeso
```

```
        ojo.recibir(95);
        bascula.recibir(32000);
        termo.recibir(43.5);
        anden.ciclo();

        // la báscula sale a mantenimiento y el motor se calienta
        Sensor fuera = anden.retirar("S2");
        ojo.recibir(150);
        termo.recibir(48);
        anden.ciclo();

        // la báscula regresa: es el mismo objeto
        System.out.println("Regresa: " + fuera);
        anden.instalar(fuera);
        banda.encender();
        System.out.println(banda.avanzar(5));
        System.out.println(anden);
    }
}
```

```
DEBE SALIR
Instalado: Sensor S1 (100%) presencia
Instalado: Sensor S2 (80%) peso, limite 25.0
Instalado: Sensor S3 (100%) temperatura, max 45.0
Instalado: Actuador A1 (100%) banda, 2 m por ciclo
Instalado: Actuador A2 (100%) semaforo en apagado
Instalado: Actuador A3 (100%) alarma, volumen 10
== Ciclo 1 en Anden 3
  S1 mide 12.0 cm
  S2 mide 18.5 kg
  S3 mide 40.0 C
  A1 avanza 2 m, lleva 2 m
  A2 en verde
  A3 en silencio
== Ciclo 2 en Anden 3
  S1 mide 9.5 cm
  S2 mide 32.0 kg ALERTA
  S3 mide 42.0 C
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Retirado: S2
== Ciclo 3 en Anden 3
  S1 mide 15.0 cm
  S3 mide 46.5 C ALERTA
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Regresa: Sensor S2 (78%) peso, limite 25.0
Instalado: Sensor S2 (78%) peso, limite 25.0
A1 avanza 5 m, lleva 7 m
Anden 3: 3 sensores, 3 actuadores, 3 ciclos
```

Fíjate en tres cosas. En el ciclo 2, **un solo** sensor en alerta detiene todo. En el ciclo 3, la estación funciona sin la báscula. Y la báscula regresa con `78%`: es **el mismo objeto**, con la batería que gastó en los ciclos 1 y 2.

Parte 4 · Para la bitácora

1. **Sobrecarga contra sobrescritura.** Da dos ejemplos de cada una **tomados de tu código** (clase y método) y explica la diferencia con tus palabras: ¿cuál decide Java al compilar, viendo los parámetros, y cuál depende del objeto?
2. **Composición contra agregación.** ¿Por qué la `Bateria` de un sensor es composición y el sensor dentro de la `Estacion` es agregación? Contesta con la pregunta de quién

hace el `new` y con lo que pasó con `S2` en el ciclo 3.

3. **Modificadores.** Escribe en `Main` la línea `bascula.gastar(5);` y córrelo. Copia el error y explica por qué `SensorPeso` sí puede llamar a `gastar` y `Main` no. Bórrala antes de entregar.

Lo que más truena

No compila

- **`SensorPresencia is not abstract and does not override abstract method hayAlerta() in Sensor`**: a tu sensor le falta uno de los métodos abstractos del padre. Una hija concreta tiene que escribir **todos**: `medir()` y `hayAlerta()`.
- **`Sensor is abstract; cannot be instantiated`**: `Sensor` es abstracta: no hay sensores a secas, solo de presencia, de peso o de temperatura.
- **`call to this must be first statement in constructor`**: el `this(...)`, igual que el `super(...)`, tiene que ser la primera línea del constructor.
- **`senal has private access in Sensor`**: la señal es privada de `Sensor`. Las hijas la leen con `getSenal()`, que es `protected`.
- **`Semaforo is not public in actuadores; cannot be accessed from outside package`**: a la clase le falta `public`. Sin modificador solo la ve su paquete, y `Main` vive en `estacion`.
- **`Bateria is not public in dispositivos; cannot be accessed from outside package`**: quisiste crear una batería desde otro paquete. La batería la crea `Dispositivo` y nadie más.
- **`leer() in SensorPeso cannot override leer() in Sensor`**: `leer()` es `final` en `Sensor`: las hijas no la sobrescriben, solo escriben `medir()` y `hayAlerta()`.
- **`cannot find symbol`**: falta un `import`. Cada clase de otro paquete se importa por su nombre completo, por ejemplo `import actuadores.Alarma;`.

Compila, pero la salida no es la de la guía

- **En el ciclo 3 vuelve a salir `S2` y falta `S3`**: tu `retirar` resta el contador pero no recorre los sensores que siguen un lugar a la izquierda.
- **En el ciclo 2 la banda sigue avanzando**: tu `ciclo()` pregunta por la alerta dentro del mismo `for` de los actuadores, o se queda solo con la del último sensor. Primero se leen **todos** los sensores, luego reaccionan los actuadores.
- **Sale `Regresa: null`**: tu `retirar` borra el sensor antes de guardarlo en una variable para devolverlo.
- **La alarma dice `volumen 15`**: el constructor de `Alarma` no valida.

La entrega, paso a paso

```
git checkout entregas_apellido_nombre
git pull origin main
mkdir -p entregas/apellido_nombre/p08
# copia aquí tu carpeta src (con sus cuatro paquetes), la carpeta out
y la bitácora en PDF
git add entregas/apellido_nombre/p08
git status
git commit -m "P8: herencia, segunda parte"
git push
```

En IntelliJ, los `.class` quedan en `out/production/`, también separados por paquete. Súbelos todos. Si no aparecen en `git status`, usa `git add -f`.

Modelos de Programación Orientada a Objetos 2027-1 · Grupo 6 · Facultad de Ingeniería, UNAM

Prof. Oscar Manuel Ruiz Hurtado · Práctica 8 · Herencia, segunda parte (MADO-21, guía 8: constructores sobrecargados, `this` y `super`, sobrecarga y sobrescritura) · octubre de 2026